1) Backtracking search.

Given: three doctors (D_1, D_2, D_3) and three shifts (Morning, Afternoon, Night)

constraints:

* D_1 cannot work night.
* D_2 must work before D_3.
* only one doctor per shift
* D_3 cannot work morning
* Each doctor must recive exactly one shift

Step-by-step solution:

* Assign D_1 to morning
* Assign D_2 to Afternoon.
* Assign D_3 to Night
* check constraints

D_1 = Morning : valid

D_2 = Afternoon comes before D_3 = Night : valid

one doctor per shift: valid

D_3 is not assigned Morning : valid

All doctors have exactly one : valid

shift	Doctor
Morning	D_1
Afternoon	D_2
Night	D_3

2) Given:

Grid size: 5x5

Start S, goal: G

Allowed moves: up, down, left, right

Each moves cost 1

obstacles cannot be crossed.

Heuristic: Manhattan distance.

Manhattan distance:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

BFS procedure:

* Insert the start node S into the queue
* Remove the first node from the queue
* Expand its unvisited neighbouring cells in the order up, down, left, right
* Ignore obstacles and cells already visited. Add neighbours to the queue and record their parents.
* Repeat until the goal G is reached.
* Trace the parent links backward from G to S to obtain the shortest path

cost calculation:

Path cost = no. of moves x 1

BFS finds the shortest path when every moves has equal cost. Manhattan distance is useful as a heuristic, but standard BFS does not use it to prioritize nodes

2) Given:

Grid size: 5x5

Start S; goal: G

Allowed moves: up, down, left, right

Each moves cost 1

obstacles cannot be crossed.

Heuristic: Manhattan distance.

Manhattan distance:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

BFS procedure:

* Insert the start node S into the queue
* Remove the first node from the queue
* Expand its unvisited neighbouring cells in the order up, down, left, right
* Ignore obstacles and cells already visited. Add neighbours to the queue and record their parents.
* Repeat until the goal G is reached.
* Trace the parent links backward from G to S to obtain the shortest path

cost calculation:

Path cost = No. of moves x 1

BFS finds the shortest path when every moves has equal cost. Manhattan distance is useful as a heuristic, but standard BFS does not use it to prioritize nodes

3) (a) Analyze the constraints:

The robot must reach a survivor at goal G from start S in a disaster-affect building

* Movement: It can move up, down, left and right.

* obstacles: Blocked cells must be avoided.

* Limited sensing: It can observe only adjacent cells.

* Movement cost: Each normal cost 1.

* Risky zone: Entering a risky zone adds 2, making the total cost of that 3.

* Dynamic environment: The robot must now update its knowledge as it discovers information.

Effect:

These constraints require safe navigation, continuous observation and cost efficient decision making.

(b) Uniform cost search (UCS)

* Start at S cost 0

* observe adjacent cells and identify obstacles and risky zone.

* Add valid neighboring cells to a priority queue according to their accumulated path cost.

* Select the cell with the lowest cost

* Update the cost when new information is discovered.

* Update the cost when new information is discovered.

* Repeat until the robot reaches G.

* Trace the path and calculate the total cost.

Cost formula:

Total cost = P + 2R

where UCS finds the minimum cost path when movement cost are non-negative, and relevant map and cost are known.

(c) Applications of AI concepts:

* Intelligent agents
* Rationality
* Environment type
* Learning and updating

(d) * It considers different movement costs It prioritizes the least cost path and can avoid expensive risky routes complete under standard conditions with branches with sensing and replanning, supports safe navigation